

POS Flow Testing — Prompt 8

Objetivo

Validar el flujo completo del POS desde login hasta cierre de caja, identificando y corrigiendo problemas de UX, errores silenciosos, race conditions y duplicación de pagos.

Flujo completo del POS

Login

Seleccionar sucursal

Abrir caja

Crear orden

Agregar productos

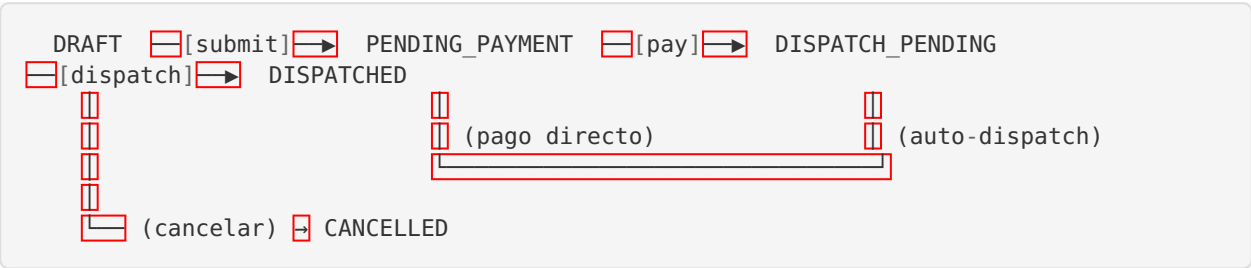
Enviar a caja

Cobrar

Despachar

Cerrar caja

Diagrama de estados de una orden



Componentes principales

Componente	Archivo	Rol
POS ventas	<code>src/components/pos/branch-pos.tsx</code>	Búsqueda de productos, ticket de venta, envío a caja
Panel de caja	<code>src/components/cash-session/cash-session-panel.tsx</code>	Abrir/cerrar sesión de caja
Cobro	<code>src/components/payments/cashier-payments.tsx</code>	Cola de cobro, procesamiento de pagos
API ventas	<code>src/app/api/sales/orders/**</code>	CRUD de órdenes y líneas
API pagos	<code>src/app/api/cashier/payments/route.ts</code>	Registro de pagos
API caja	<code>src/app/api/cashier/cash-sessions/**</code>	Gestión de sesiones de caja
Servicio ventas	<code>src/modules/sales/service.ts</code>	Lógica de negocio de ventas
Servicio pagos	<code>src/modules/payments/service.ts</code>	Lógica de negocio de pagos

Problemas encontrados y corregidos

● Críticos

1. Llamadas `fetch()` crudas sin CSRF token (branch-pos.tsx)

- **Problema:** Todas las operaciones POST/PATCH/DELETE en `branch-pos.tsx` usaban `fetch()` directo, sin incluir el header `x-csrf-token`. Esto causaba que todas las operaciones mutantes fallaran con HTTP 403 si CSRF estaba habilitado.
- **Corrección:** Reemplazado `fetch()` por `apiFetch()` de `@lib/client/api` en todas las operaciones mutantes (agregar producto, actualizar cantidad, eliminar línea, enviar a caja).
- **Archivos:** `src/components/pos/branch-pos.tsx`

2. Llamadas `fetch()` crudas sin CSRF token (cashier-payments.tsx)

- **Problema:** El POST de pago usaba `fetch()` crudo.
- **Corrección:** Reemplazado por `apiFetch()`.
- **Archivos:** `src/components/payments/cashier-payments.tsx`

3. Llamadas `fetch()` crudas sin CSRF token (cash-session-panel.tsx)

- **Problema:** Abrir, solicitar cierre y cerrar sesión usaban `fetch()` crudo.
- **Corrección:** Reemplazado por `apiFetch()`.
- **Archivos:** `src/components/cash-session/cash-session-panel.tsx`

4. Funciones async sin try-catch en cash-session-panel.tsx

- **Problema:** `openSession()`, `requestCloseSession()` y `closeSession()` no tenían try-catch. Un error de red dejaba `busyAction` atascado permanentemente, bloqueando toda interacción.
- **Corrección:** Envueltas en try-catch-finally. El `finally` siempre resetea `busyAction(null)`.
- **Archivos:** `src/components/cash-session/cash-session-panel.tsx`



Importantes

5. Doble clic puede duplicar pagos (cashier-payments.tsx)

- **Problema:** Si el usuario hacía doble clic rápido en “Cobrar”, la segunda llamada podía iniciar antes de que `isSubmitting` se activara.
- **Corrección:** Agregado `recentlyPaidRef` (Set de orderId) como guardia adicional. La orden se añade al set inmediatamente y se remueve después de 3 segundos.
- **Archivos:** `src/components/payments/cashier-payments.tsx`
- **Nota:** El backend ya tiene protección via `FOR UPDATE` + verificación de `PAYMENT_ALREADY_POSTED`, pero la guardia en UI evita llamadas innecesarias.

6. Hotkey Enter podía disparar pago con estado stale

- **Problema:** El event listener de hotkeys capturaba `canSubmitPayment` en una closure, que podía estar desactualizada.
- **Corrección:** Uso de refs (`canSubmitRef`, `isSubmittingRef`) sincronizadas con efectos.
- **Archivos:** `src/components/payments/cashier-payments.tsx`

7. Pantalla muda al cargar POS (branch-pos.tsx)

- **Problema:** Al cargar el POS, se mostraba “preparando...” sin spinner ni indicación clara de carga.
- **Corrección:** Agregado estado `isInitialLoading` con spinner animado y texto “Preparando punto de venta...”.
- **Archivos:** `src/components/pos/branch-pos.tsx`

8. Notificaciones no se auto-ocultaban

- **Problema:** Las notificaciones (notice) permanecían en pantalla indefinidamente.
- **Corrección:** Implementado `setNoticeTimed()` con auto-dismiss (6s para éxito, 10s para errores). Se limpia el timer al desmontar.
- **Archivos:** `src/components/pos/branch-pos.tsx`



Mejoras menores

9. Respuestas 403 sin campo reason

- **Problema:** `toHttpErrorResponse` retornaba 403 para errores de autorización sin el campo `reason`, dificultando el mapeo de mensajes en el cliente.
- **Corrección:** Agregado `reason: error.message` a todas las respuestas 403 de autorización.
- **Archivos:** `src/lib/http.ts`

10. Traducciones de error incompletas en pos-ui.ts

- **Problema:** Faltaban traducciones al español para varios códigos de error del backend.
- **Corrección:** Agregados 13 nuevos mapeos de error.
- **Archivos:** `src/lib/pos-ui.ts`
- **Códigos agregados:** `NO_ACTIVE_CASH_BOX`, `CASH_SESSION_ALREADY_OPEN`, `CASH_SESSION_CASH_BOX_INVALID`, `CASH_SESSION_NOT_RECONCILING`, `CASH_SESSION_UNRESOLVED_ORDERS`, `CASH_SESSION_HAS_PENDING_PAYMENTS`,

CASH_SESSION_DISCREPANCY_REQUIRES_APPROVAL , CASH_BOX_INACTIVE , CASH_BOX_BRANCH_MISMATCH ,
CASHIER_MODULE_ENABLED , INVALID_TRANSITION , INVALID_PAYMENT_AMOUNT

Protecciones contra doble pago

El sistema tiene **tres capas** de protección:

1. **UI (cashier-payments.tsx):** `isSubmitting` + `recentlyPaidRef` (Set temporal de orderId)
2. **Backend (payments/service.ts):** `SELECT ... FOR UPDATE` en `SaleOrder` bloquea filas durante la transacción
3. **Base de datos:** Verificación de `PAYMENT_ALREADY_POSTED` + constraint P2002

Scripts de verificación

Script	Comando	Descripción
<code>verify-sales.mjs</code>	<code>npm run verify:sales</code>	Valida 42 checks del flujo de ventas
<code>verify-payments.mjs</code>	<code>npm run verify:payments</code>	Valida 53 checks del flujo de pagos
Typecheck	<code>npm run typecheck</code>	Compilación TypeScript sin errores

Ejecución

```
npm run verify:sales      # 42 checks ✓
npm run verify:payments  # 53 checks ✓
npm run typecheck        # Sin errores ✓
```

Guía de testing manual para QA

Prerrequisitos

1. Base de datos PostgreSQL con datos de prueba (seed)
2. Al menos un usuario con rol `CASHIER` o `BRANCH_ADMIN`
3. Productos activos con stock disponible
4. Al menos una caja física activa en la sucursal

Escenarios de prueba

E1: Flujo completo exitoso

1. Iniciar sesión con usuario cajero
2. Ir a POS → verificar que muestra spinner “Preparando punto de venta...”

3. Seleccionar caja física → Ingresar monto de apertura → Clic “Abrir sesión”
4. Verificar mensaje “Sesión abierta correctamente. ✓”
5. Buscar producto por nombre/SKU → Enter o clic para agregar
6. Verificar notificación “Producto agregado: [nombre]”
7. Modificar cantidad → clic “Aplicar” → verificar actualización
8. Clic “Enviar a caja” → verificar “Orden enviada a caja”
9. Ir a pantalla de cobro → verificar que la orden aparece en cola
10. Seleccionar orden → Seleccionar método de pago → Clic “Cobrar”
11. Verificar toast “Pago aplicado correctamente”
12. Solicitar cierre de caja → Ingresar monto de cierre → Cerrar
13. Verificar “Sesión cerrada correctamente. ✓”

E2: Errores de red

1. Desconectar red/simular offline
2. Intentar agregar producto → verificar mensaje “Error de red”
3. Intentar enviar a caja → verificar mensaje de error
4. En panel de caja, intentar abrir sesión → verificar que busyAction se resetea

E3: Stock insuficiente

1. Intentar agregar cantidad mayor al stock disponible
2. Verificar mensaje “Stock insuficiente. Disponible: X.XX”
3. Intentar enviar orden con producto sin stock
4. Verificar mensaje “Stock insuficiente para completar la operación”

E4: Doble pago

1. En cobro, hacer doble clic rápido en “Cobrar”
2. Verificar que solo se procesa un pago
3. Verificar toast “Esta orden ya se está procesando” en el segundo clic

E5: Sesión de caja cerrada

1. Sin sesión de caja abierta, intentar cobrar
2. Verificar toast “No puedes cobrar sin una sesión de caja abierta”

E6: CSRF token expirado

1. Esperar a que expire el CSRF token (o invalidarlo manualmente)
2. Realizar operación mutante
3. Verificar que `apiFetch` renueva el token automáticamente y reintenta

E7: Permisos insuficientes

1. Iniciar sesión con usuario sin permisos de caja
2. Verificar mensaje “Tu rol no tiene permisos para esta operación”

E8: Transporte

1. Activar checkbox “Requiere transporte” → dejar monto vacío
2. Verificar error “El transporte está activado, pero falta el monto”
3. Ingresar monto válido → enviar a caja
4. Verificar que el total incluye transporte en la pantalla de cobro

E9: Cierre con discrepancia

1. Solicitar cierre de caja

2. Ingresar monto de cierre con diferencia > C\$5
3. Verificar mensaje “Solicitud enviada. Un aprobador debe validar...”

Archivos modificados

Archivo	Tipo de cambio
src/components/pos/branch-pos.tsx	CSRF (apiFetch), loading state, auto-dismiss, feedback
src/components/payments/cashier-payments.tsx	CSRF (apiFetch), doble-clic guard, hotkey fix
src/components/cash-session/cash-session-panel.tsx	CSRF (apiFetch), try-catch-finally
src/lib/pos-ui.ts	13 nuevas traducciones de error
src/lib/http.ts	campo reason en 403
scripts/verify-sales.mjs	Nuevo script de verificación
scripts/verify-payments.mjs	Nuevo script de verificación
package.json	Scripts verify:sales y verify:payments actualizados

Archivos NO modificados (prohibidos)

- ❌ Inventario (src/modules/inventory/**)
- ❌ Schema Prisma (prisma/schema.prisma)
- ❌ Estilos/diseño visual